



universität
uulm

Interactive Computer Graphics

Chapter 1 – Introduction

Summer Term 2026

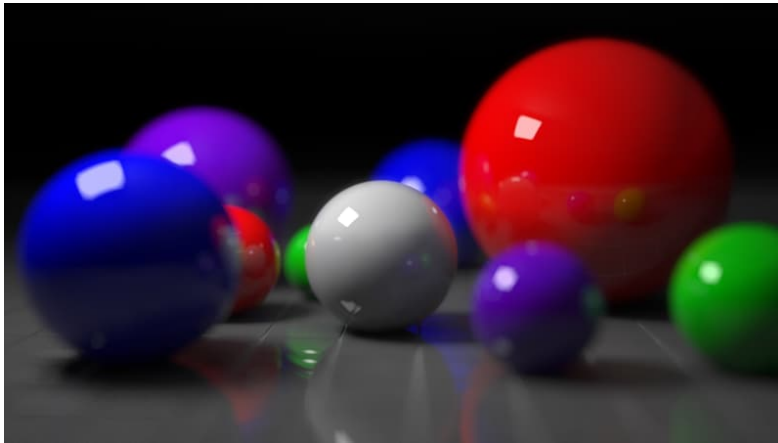
Timo Ropinski



Visual Computing Group
Institute of Media Informatics

Course Learning Outcomes

- Understand theoretical foundations of interactive computer graphics
- Learn to program computer graphics applications using OpenGL
- Learn to realize efficient computer graphics applications on the GPU
- Implement your own computer graphics project



[Image Credits: Wikipedia User Mimigu]



[Crassin et al., PG2011]

Real-Time Rendering Contest Results 1/3



Students: Frank Richter, Manuel Herbert Güntzel

Real-Time Rendering Contest Results 2/3

Render Parameters

Framerate: 39 FPS

General Settings

☐ Enable Phong Lighting

☐ Enable Normalmapping

☐ Enable Flashlights

Shadow Settings

☐ Enable Shadow-Mapping

☐ No smooth shadows

☐ Fast smooth shadows

☐ slow smooth shadows

Water Settings

☐ Enable Water

☐ Small waves

☐ Middle waves

☐ High waves

Postprocessing

☐ Enable Postprocessing

☐ Gaussian Filter

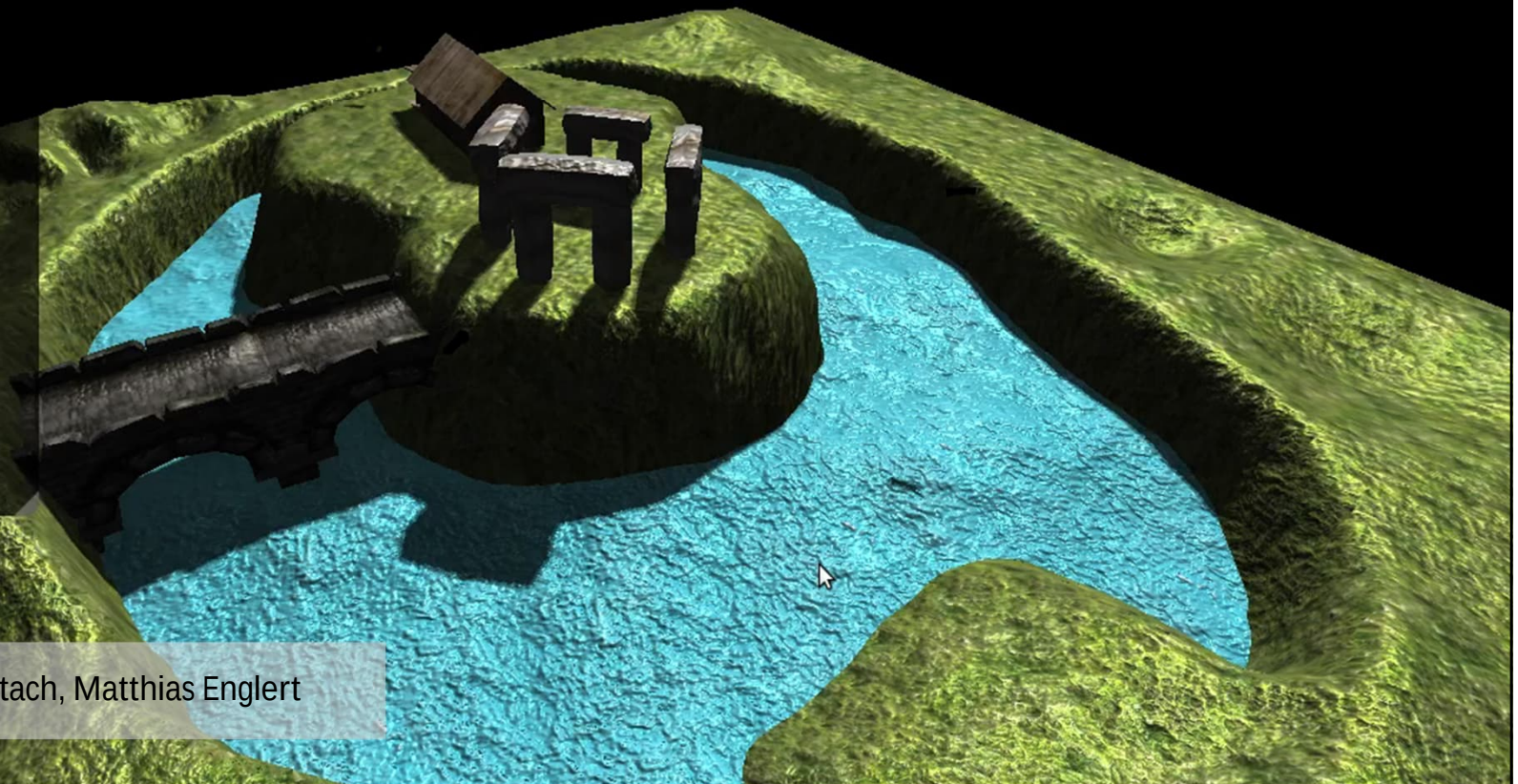
☐ Sobel Filter

☐ Sharp Filter

☐ RGB-Max Filter

☐ Intensity-Max Filter

Students: Andreas Rottach, Matthias Englert



Real-Time Rendering Contest Results 3/3





universität
uulm

1.1 Course Organization

Details about the lecture course



Visual Computing Group
Institute of Media Informatics

Teachers

- Lectures

Timo Ropinski

timo.ropinski@uni-ulm.de

Office hours upon request

- Exercises

Poonam

poonam.poonam@uni-ulm.de

Office hours upon request

Aditya Sawant

aditaya.sawant@uni-ulm.de

Course Format

- Lectures

- Tuesday (14-16) and every other Wednesdays (12-14)
 - Room O28/1002
 - Specific dates/modalities in Moodle (password: ICG26SUMMER)
- Lecture slides
 - Available before lecture in Moodle
 - Will be in general not self-explanatory

Course Format

- Exercises
 - Every other Wednesday (12-14)
 - Room O28/1002
 - Programming in C/C++ (introduction available upon request)
 - Enroll in Moodle

Exercises

- Exercise
 - Exercise 1 – Rendering Pipeline
 - Exercise 2 – Primitive Types and Shaders
 - Exercise 3 – Geometric Transformations
 - Exercise 4 – Projection and Clipping
 - Exercise 5 – Rasterization
 - Project Exercise – Real-Time Rendering Contest
- Exercise sheets cover theory and practice
 - New exercises released on Tuesday evenings 6pm
 - Hand in by another Tuesday evenings 23:55pm
- Exercise solutions to be submitted by teams of two students

Successful Course Completion

- Pass the oral/written exam
 - Dates to be announced
- „Notenbonus“ possible
 - 50% success rate of exercise solutions for n-1 sheets
 - Only running solutions are marked
- Presentation of the solution for one exercise
 - Detailed derivation of the solution
 - Addressing questions of fellow students

Course Overview

- Chapter 1 – Introduction
- Chapter 2 – The Rendering Pipeline
- Chapter 3 – Geometric Transformations
- Chapter 4 – Geometric Projection
- Chapter 5 – Clipping
- Chapter 6 – Rasterization
- Chapter 7 – Visibility Determination
- Chapter 8 – Illumination and Shading
- Chapter 9 – Texturing
- Chapter 10 – Real-Time Shadows
- Chapter 11 – Ray Tracing

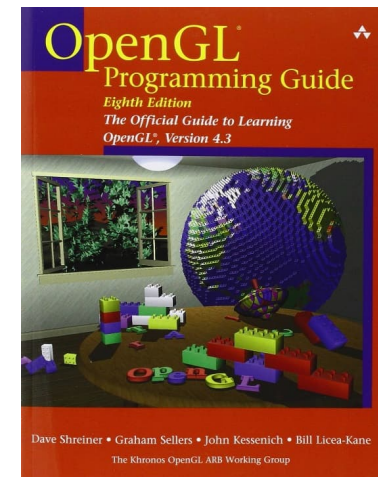
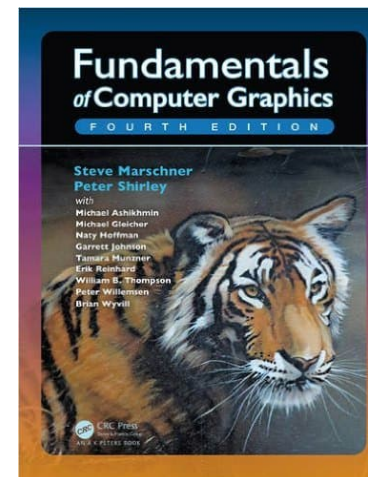
Background

Journey along the
Rendering Pipeline

Lighting Effects

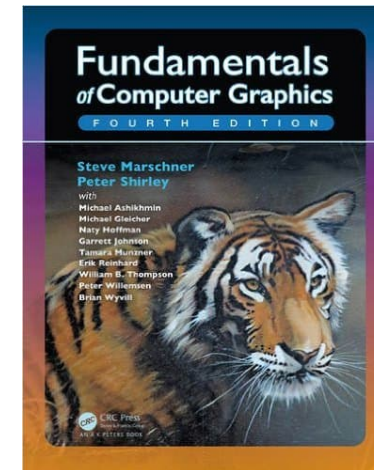
Literature

- P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer Graphics (4th Edition), AK Peters 2016.
- D. Shreiner, G. Sellers, J. Kessenich, B. Licea-Kane: OpenGL Programming Guide: The Official Guide to Learning OpenGL (8th Edition), Addison-Wesley 2013.
- References provided at end of each chapter



Relevant Chapters and Link to Lecture Course

- Chapter 1: Introduction
 - Chapter 3: Raster Images
 - Chapter 19: Color
 - Chapter 8: The Graphics Pipeline (Our Chapter 2)
 - Chapter 6: Transformation Matrices (Our Chapter 3)
 - Chapter 7: Viewing (Our Chapter 4)
 - Chapter 10: Surface Shading (Our Chapter 8)
 - Chapter 11: Texture Mapping (Our Chapter 9)
 - Chapter 4: Ray Tracing (Our Chapter 11)
-
- For self study
 - Chapter 2: Miscellaneous Math
 - Chapter 5: Linear Algebra



Chapter Outline

(1.1 Course Organization)

1.2 Computer Graphics Overview

1.3 Pixel-based Representations

1.4 3D Models

1.5 Algorithmic Paradigms

References



universität
uulm

1.2 Computer Graphics Overview

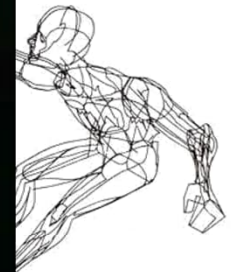
Relation of the field with respect to other disciplines



Visual Computing Group
Institute of Media Informatics

Comp

- Origin
 - Pe
- Comp
 - 19
 - 19
 - 19
 - [at](#)
 - 19
 - 19
 - 19
 - 19
 - 19

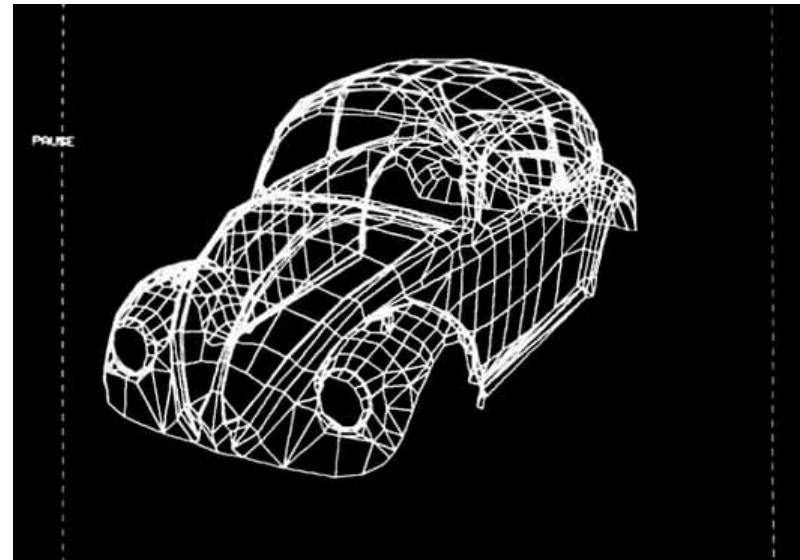


EXPANDED ENTERTAINMENT

Computer Graphics History

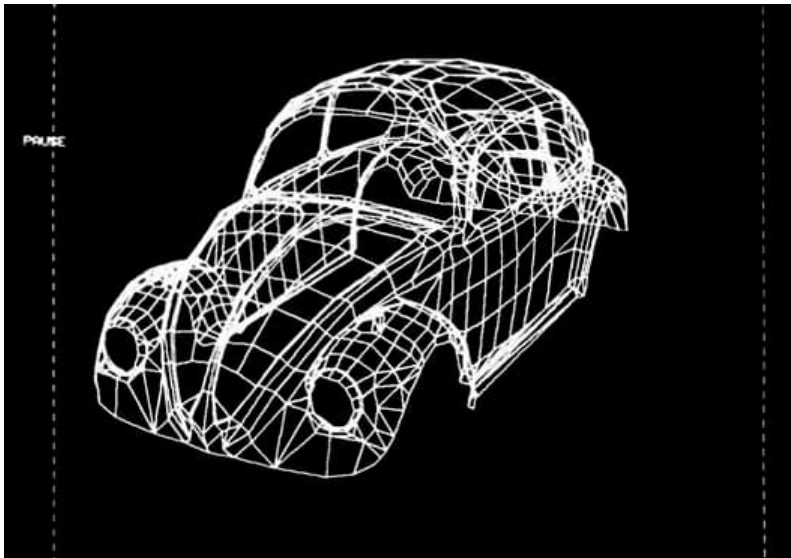


Sutherland, Utah (1970)



Wireframe

Computer Graphics History



Sutherland, Utah (1970)



Turner Whitted, Siggraph (1979)

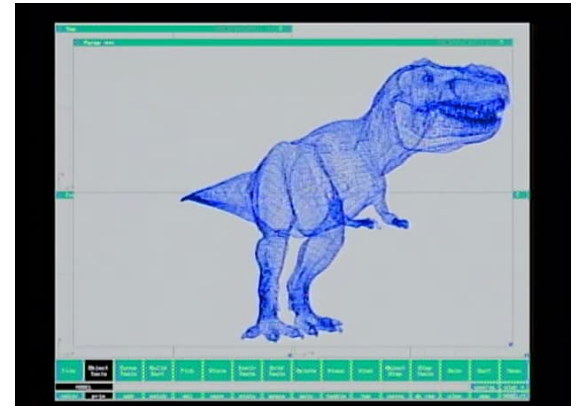
Computer Graphics History



The Abyss (1989)



Terminator 2 (1991)



Jurassic Park (1993)

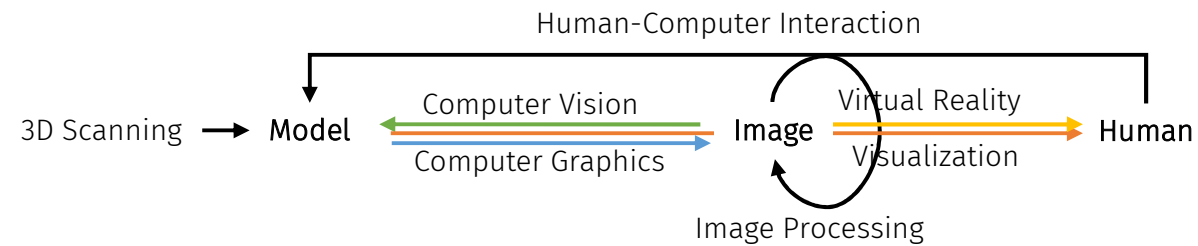
Computer Graphics History



Ikea (2014)

Related Disciplines

- Human-Computer Interaction
- Virtual and Augmented Reality
- Visualization
- Image Processing
- Computer Vision
- 3D Scanning



Applications

- Computer games
- Animation movies
- Special effects in movies
- CAD/CAM
- Simulation



[CryEngine 5.7, CryTek 2022]



[Terminator 2, Tri-Star Pictures 1991]



[Toy Story, Pixar 1995]



[Avatar: Way of Water, 20th Century 2022]



universität
uulm

1.3 Pixel-based Representations

What is a pixel?

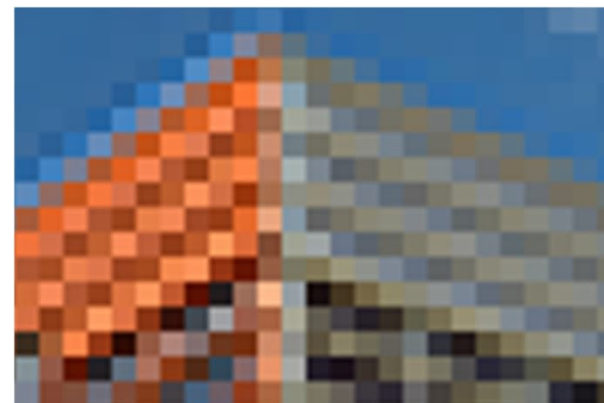
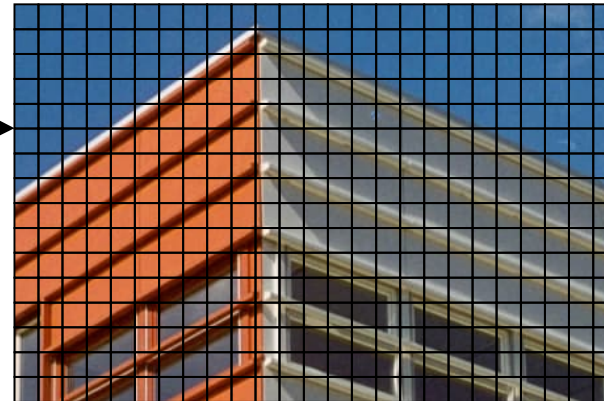


Visual Computing Group
Institute of Media Informatics

Raster Images

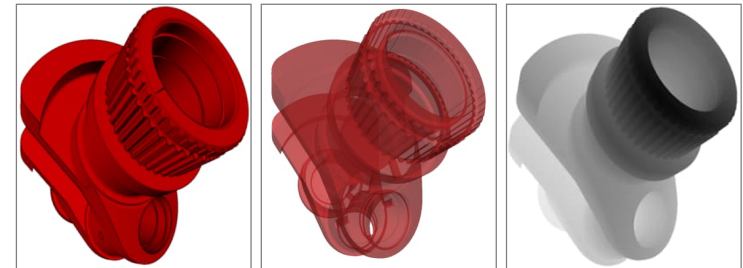


[„Bibliothek“ by press office Ulm University]



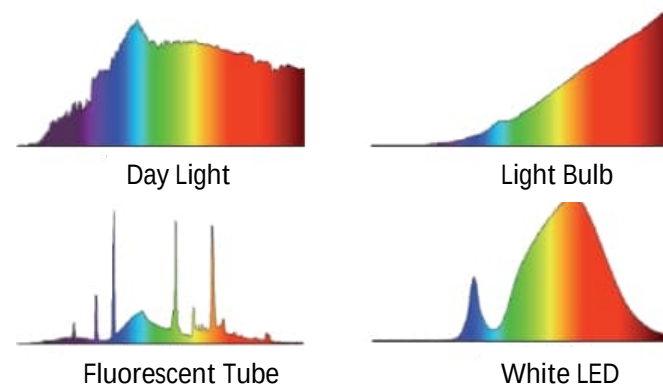
Pixel

- Name derived from Word for Picture Element
- Pixel is a sample with associated sample values (=pixel components)
 - Color
 - Transparency (=alpha value)
 - Depth (i.e., distance from viewer (=z value))
 - ...
- Each sample value stored at given precision (bit depth)
 - e.g. 24 Bit color, 8 Bit alpha + 24 Bit depth

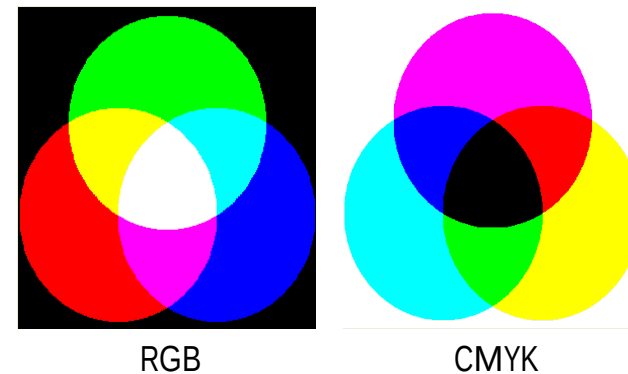


Color

- In real world color is given by color spectrum

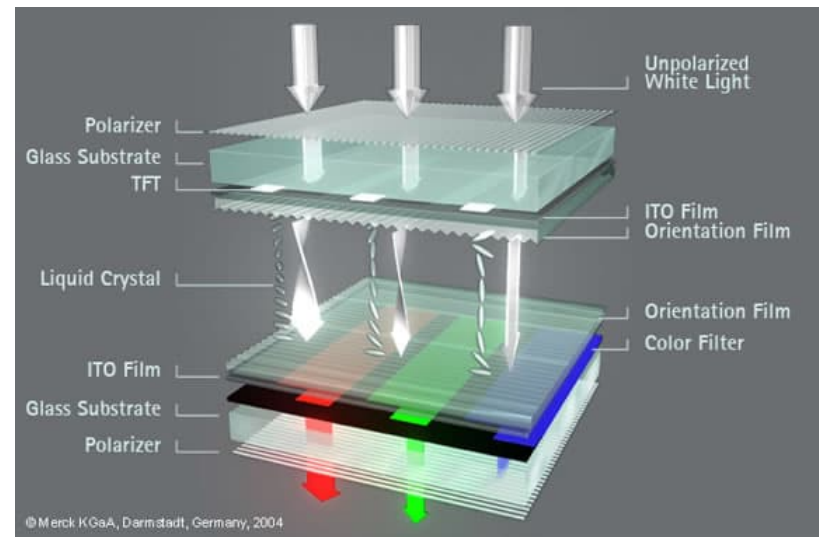
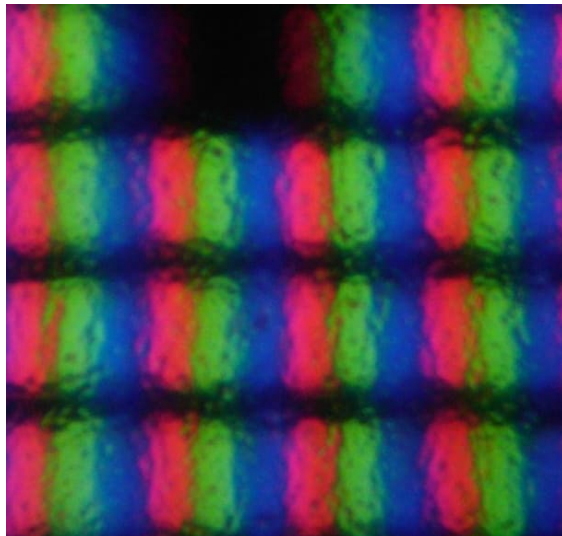


- In computer graphics color is represented through color models
 - RGB color model (additive)
 - CMYK color model (subtractive)
 - HSV color model (user-centered)



LCD Raster Screen

- Liquid crystal displays exist since the 90s and enable flat screens
- Liquid crystals affect the polarization of light
- Based on polarization, subpixel visibility is regulated



Raster Image Sizes

- Assume a raster image with 1024x768 pixels and 24 Bit color depth
 - How many different colors can be represented?
 - $2^{24} = 2^8 \cdot 2^8 \cdot 2^8 = 16.777.216$
 - How much memory does the image need?
 - $1024 \cdot 768 \cdot 24 = 18.874.368b = 2.359.296B = 2,25MiB$
 - How long is each image visible at a frame rate of 30Hz?
 - $\frac{1.000ms}{30} = 33,33ms$
 - How much image data needs to be generated per second?
 - $2,25MiB \cdot 30 = 67,5MiB$



universität
uulm

1.4 3D Models

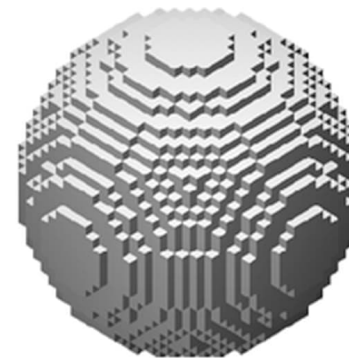
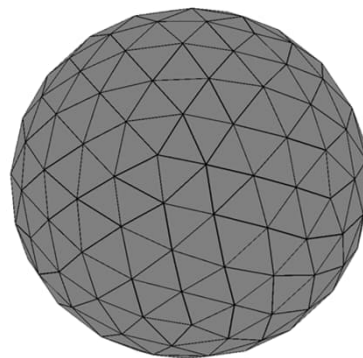
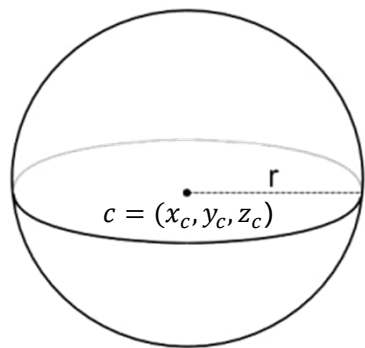
Modeling with triangles and other geometric primitives



Visual Computing Group
Institute of Media Informatics

3D Model Types

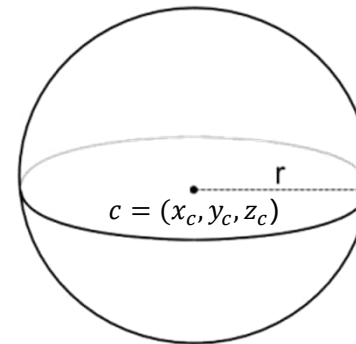
- Models specify the structures to be displayed
- Three commonly used model types exist
 - Implicit surfaces
 - Polygonal models
 - Volumetric models



$$f(x, y, z) = (x - x_c)^2 + (y - y_c)^2 + (z - z_c)^2 - r^2$$

Implicit Surfaces

- Set of all points that fulfill $f(x, y, z) = 0$
- All points with $f(x, y, z) < 0$ define solid bound by $f(x, y, z) = 0$
- Display possibilities
 - Search for null points
 - Approximate with polygons
 - Rasterize
- Not all objects can be represented through implicit surfaces

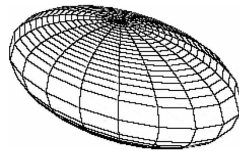


$$f(x, y, z) = (x - x_c)^2 + (y - y_c)^2 + (z - z_c)^2 - r^2$$

Quadrics

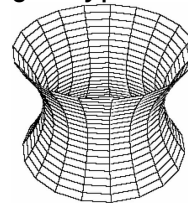
- Surfaces represented by quadric equations

Ellipsoid



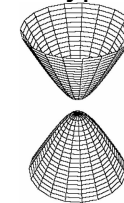
$$f(x, y, z) = \frac{x^2}{\alpha^2} + \frac{y^2}{\beta^2} + \frac{z^2}{\gamma^2} - 1$$

Single Hyperboloid



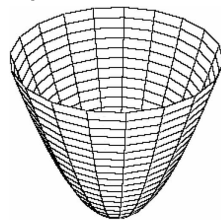
$$f(x, y, z) = \frac{x^2}{\alpha^2} + \frac{y^2}{\beta^2} - \frac{z^2}{\gamma^2} - 1$$

Double Hyperboloid



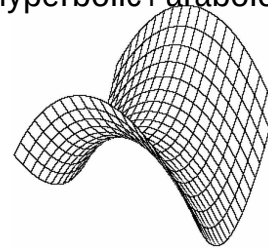
$$f(x, y, z) = \frac{x^2}{\alpha^2} + \frac{y^2}{\beta^2} - \frac{z^2}{\gamma^2} + 1$$

Elliptic Paraboloid



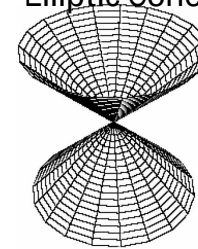
$$f(x, y, z) = \frac{x^2}{\alpha^2} + \frac{y^2}{\beta^2} - z$$

Hyperbolic Paraboloid



$$f(x, y, z) = -\frac{x^2}{\alpha^2} + \frac{y^2}{\beta^2} - z$$

Elliptic Cone



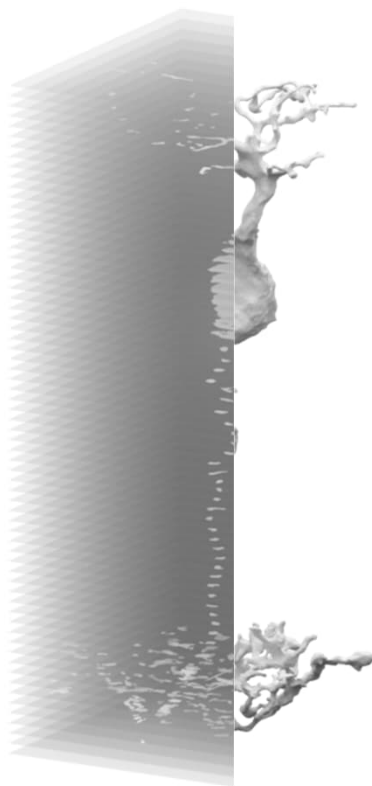
$$f(x, y, z) = \frac{x^2}{\alpha^2} + \frac{y^2}{\beta^2} - z^2$$

[Eldar Sultanow: Implicit Surfaces]

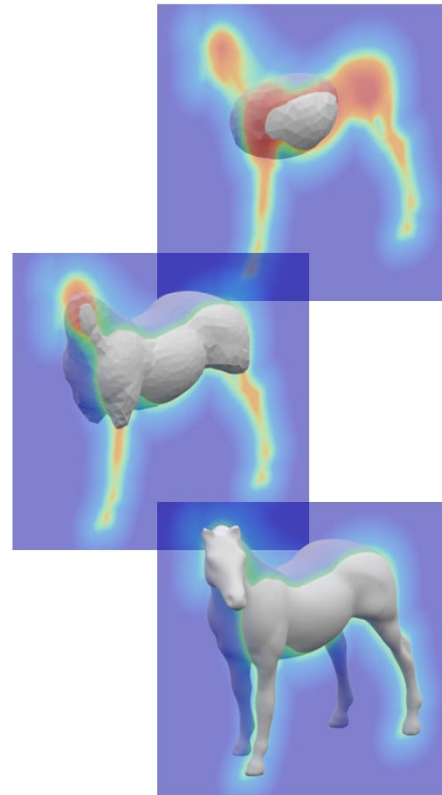
Object Representation



Point Cloud (3D Scan)



Binary Data (MRT)



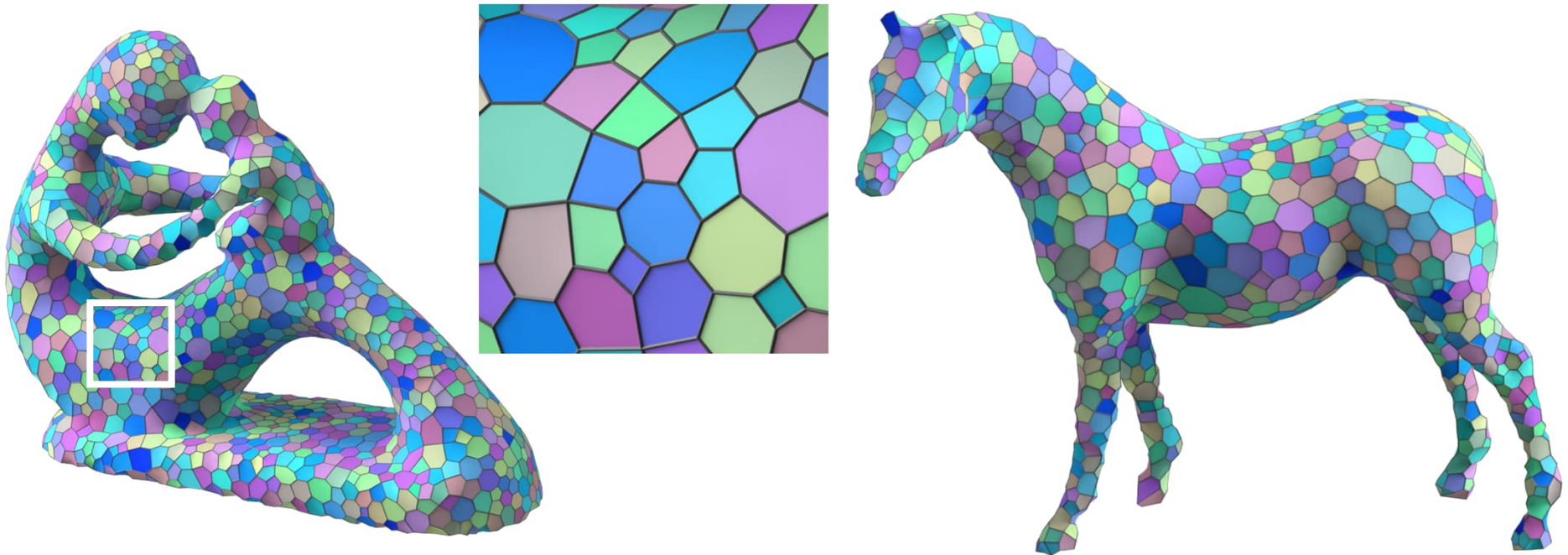
Signed Distance Fields (SDF)



Meshes

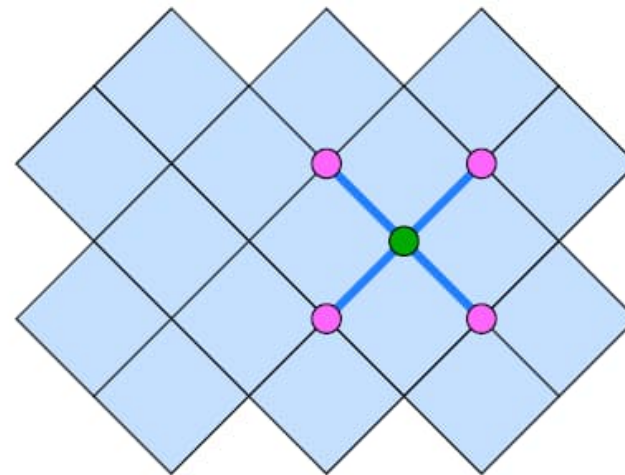
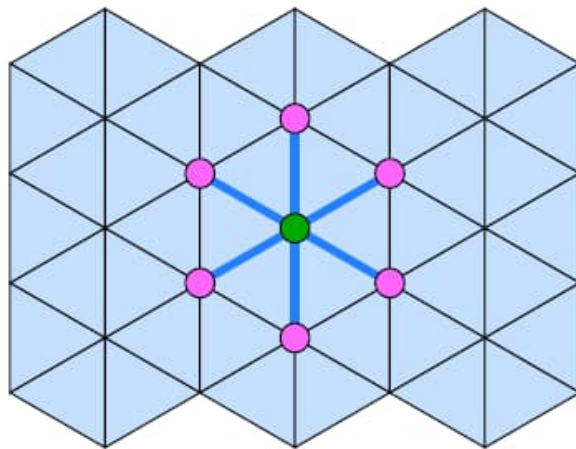
Polygonal Models 1/3

- Surface representation is approximated by polygons
 - Polygons are made up of vertices and edges



Polygonal Models 1/4

- Surface representation is approximated by polygons (mostly triangles/quads)
 - Polygons are made up of vertices and edges



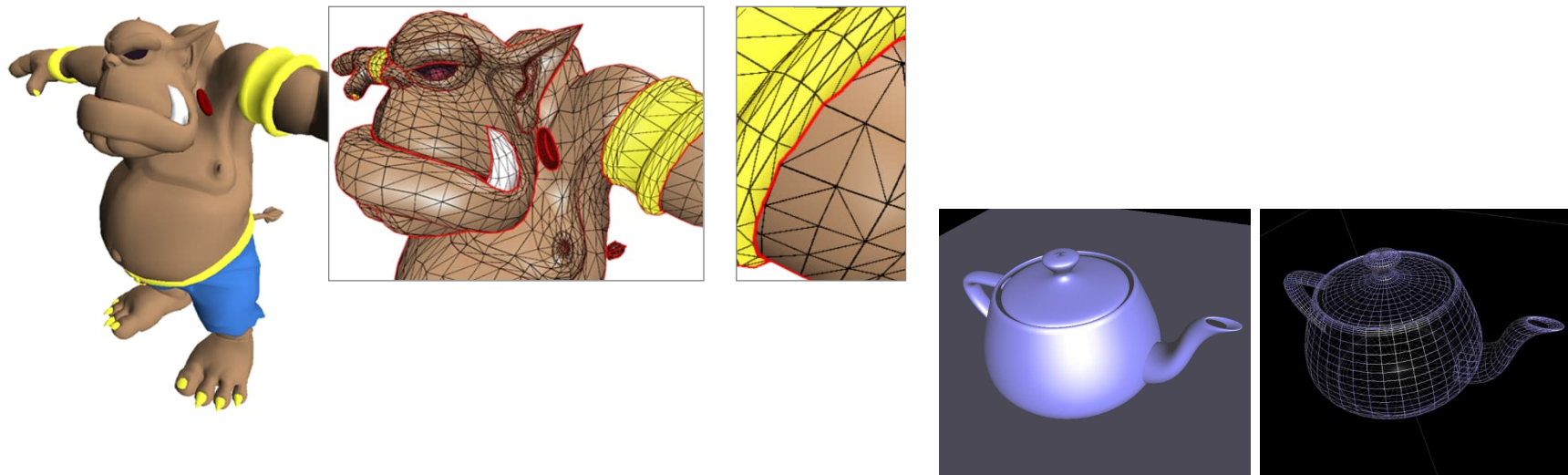
Valence / Degree

Adjacent Edges

1-Ring Neighborhood

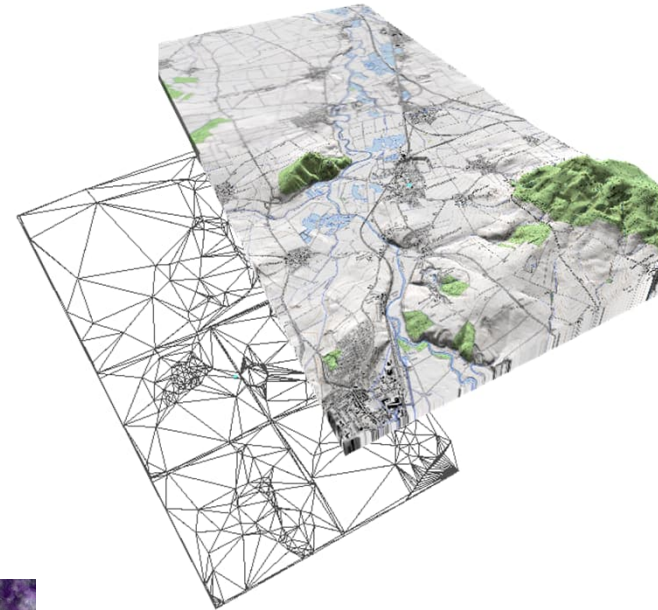
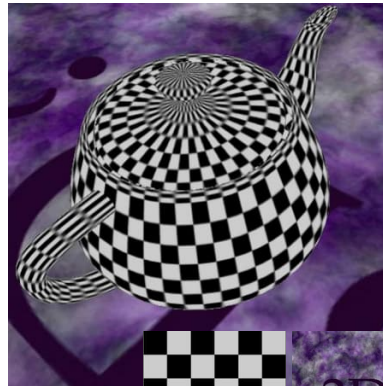
Polygonal Models 2/4

- Surface representation are often approximated by triangular polygons
 - Triangles consist of three vertices and three edges
- Display achieved through rendering by raster conversion of triangles



Polygonal Models 3/4

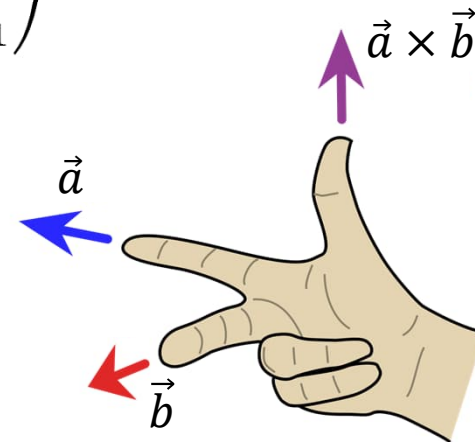
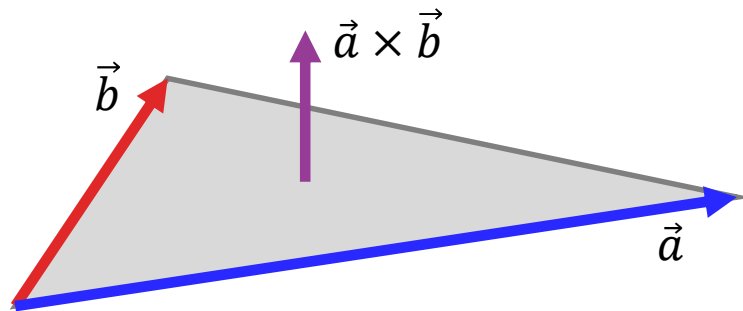
- Various attributes are associated with vertices of a polygonal model
 - Colors
 - Material properties
 - Textures
 - ...



Polygonal Models 4/4

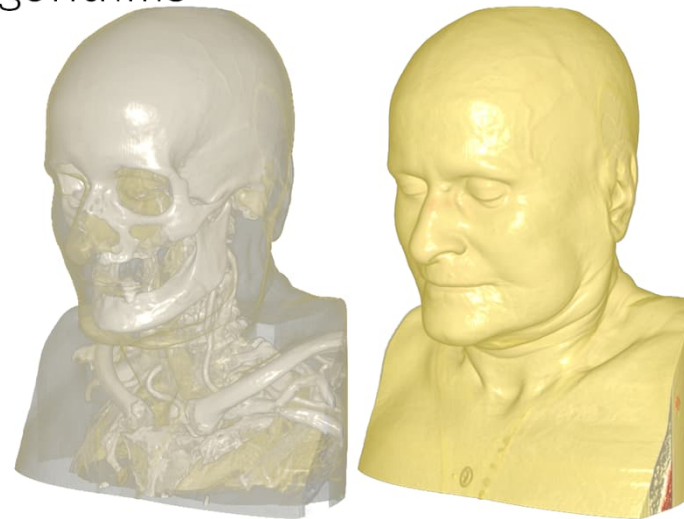
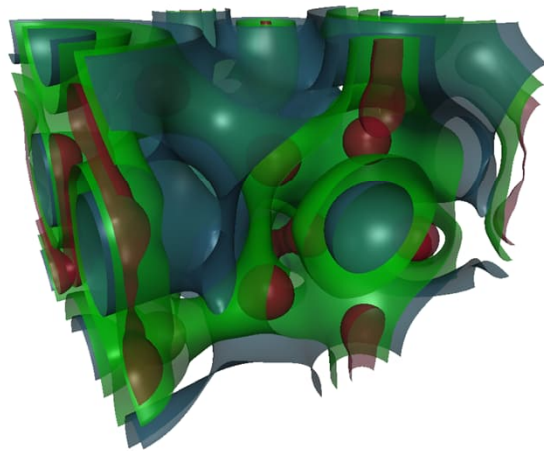
- Surface normal required in many cases
- Normal is calculated through cross product:

$$\vec{a} \times \vec{b} = \begin{pmatrix} a_2b_3 - a_3b_2 \\ a_3b_1 - a_1b_3 \\ a_1b_2 - a_2b_1 \end{pmatrix}$$



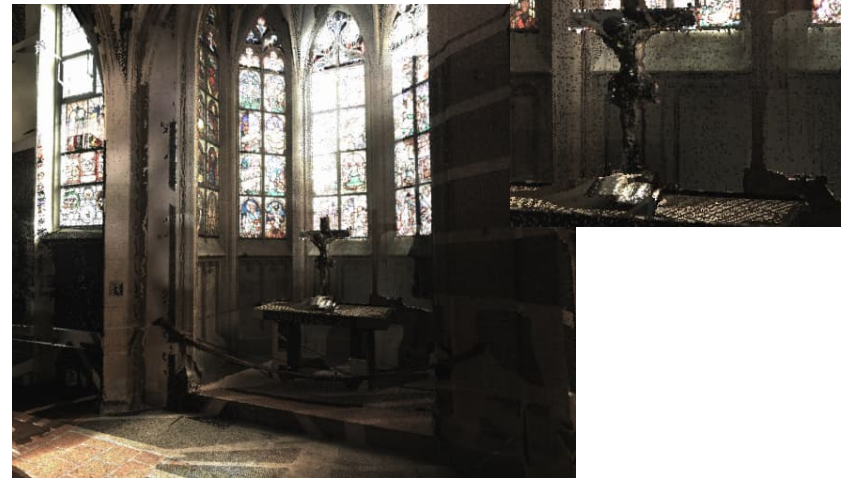
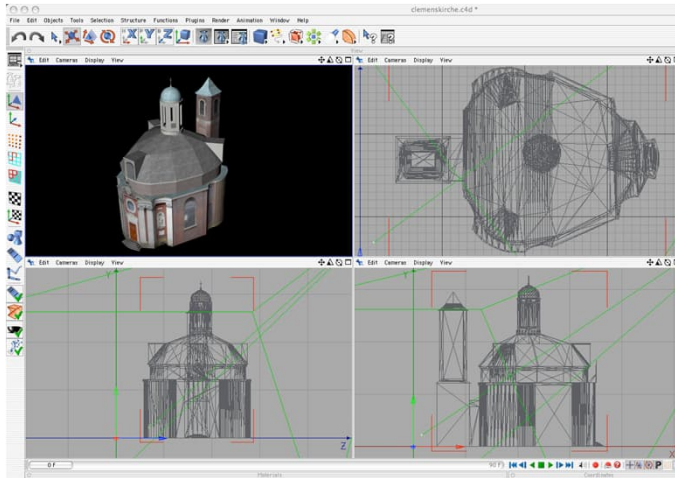
Volumetric Models

- Space filling object representation through voxels (volume elements)
- Display possibilities
 - Rendering of extracted surfaces
 - Direct rendering through specialized algorithms



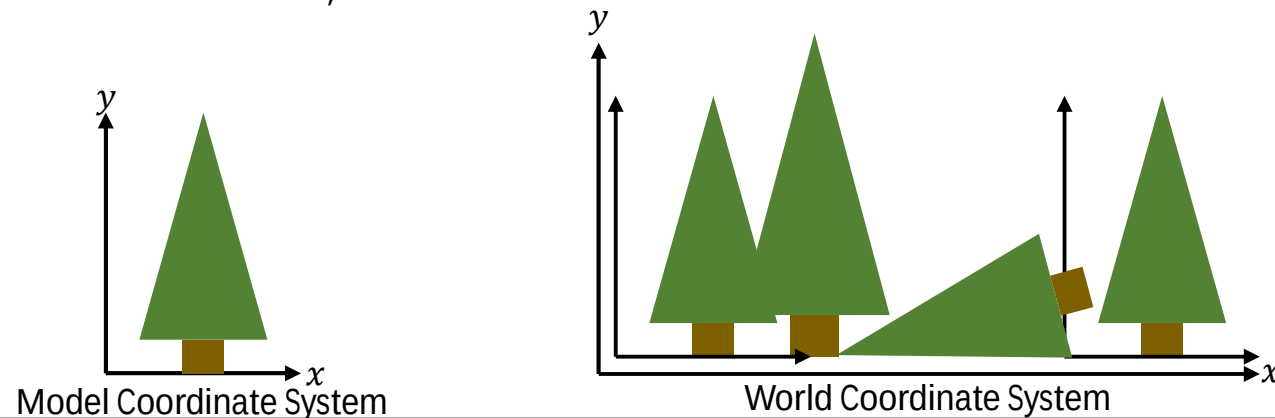
Model Generation

- Polygonal models are generated through 3D scanning or 3D modeling (e.g., LIDAR, Blender, Cinema 4D, Maya)
- Volumetric models are generated through imaging or simulation (e.g., CT, MRI, PET, CFD, seismic surveys)



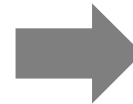
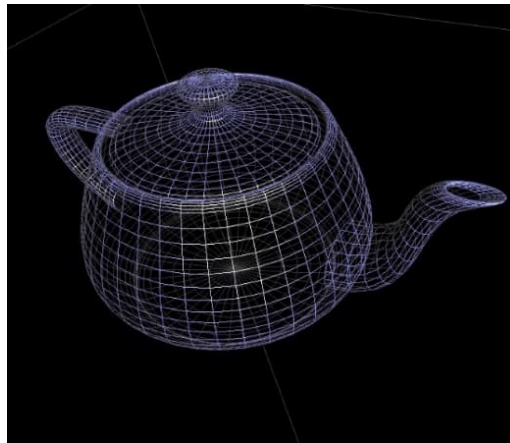
Coordinate Systems

- Three main coordinate system exist
 - Model coordinate system – model specified in own coordinate system
 - World coordinate system – models arranged in common coordinate system
 - Eye/camera coordinate system – model in relation to camera
- To transfer between coordinate systems, models can be translated, rotated or scaled



Rendering

- Rendering is the process to generate raster images from 3D models
- During rendering several steps are necessary
 - Coordinate system transformation
 - Visibility determination
 - Illumination
 - Texturing
 - ...





universität
uulm

1.5 Algorithmic Paradigms

Frequently found algorithm strategies



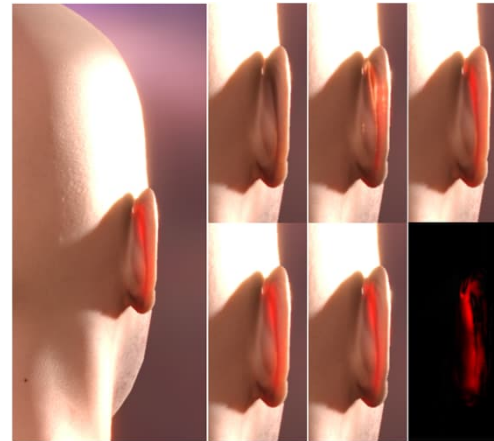
Visual Computing Group
Institute of Media Informatics

Photorealistic Rendering

- Goal: simulate reality



[Embree Ray Tracing Website - <http://embree.github.io/>]



[Maisch & Ropinski, Eurographics 2017]



[Hendrik vann Jenssen]

Non-Photorealistic Rendering (NPR)

- Goal: abstract reality



[Suggestive contour line drawings, DeCarlo 2003]

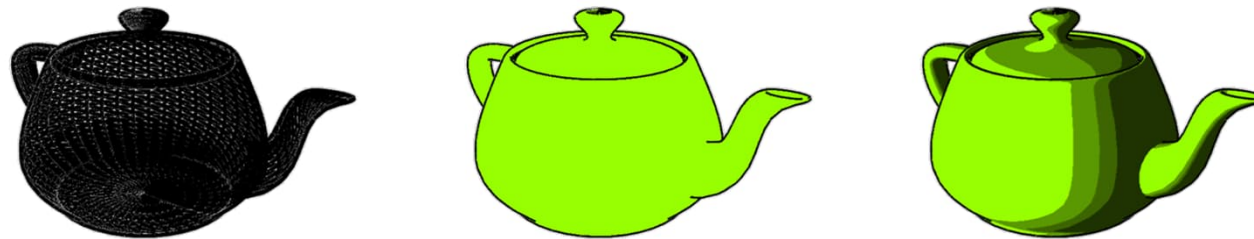


Image-Based Algorithms

- Number of computations direct proportional to number of pixels

```
for all Pixels do {  
    for all SceneObjects do {  
        ...  
    }  
}
```

- Benefits
 - Easy implementation
 - Easy realization of shadows and reflections since all objects accessible in inner loop
- Downsides
 - Time intensive computations

Object-Based Algorithms

- Number of computations direct proportional to number of 3D models

```
for all SceneObjects do {  
  for all Pixels do {  
    ...  
  }  
}
```

- Benefits
 - Fast computation
- Downsides
 - Difficult implementation
 - Difficult realization of shadows and reflections since only single object accessible in inner loop

Chapter Summary

- Raster images are formed of pixels to which colors, transparencies and depth are assigned
- In interactive computer graphics 3D models are mostly approximated by polygonal models
- Most frequent building blocks of polygonal models are triangles, which are formed of vertices and edges
- Rendering translates 3D models into raster images



universität
uulm

References

Literature and other sources used in this chapter



Visual Computing Group
Institute of Media Informatics

References

- Text Books
 - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer Graphics (4th Edition), AK Peters 2016. (Chapters 1, 3 & 21)
- Research Publications for Image References
 - Crassin, Cyril, Fabrice Neyret, Miguel Sainz, Simon Green, and Elmar Eisemann. "Interactive indirect illumination using voxel cone tracing." In Computer Graphics Forum, vol. 30, no. 7, pp. 1921-1930. Oxford, UK: Blackwell Publishing Ltd, 2011.
 - Maisch, Sebastian, and Timo Ropinski. "Spatial Adjacency Maps for Translucency Simulation under General Illumination." In Computer Graphics Forum, vol. 36, no. 2, pp. 443-453. 2017.
 - Reichardt, Jasja. "Cybernetic Serendipity. The Computer and the Arts." Studio International, London (1968).
- YouTube Videos
 - Spacewar! (MIT 1962)
 - Simulation of a two-giro gravity attitude control system - Edward Zajac
 - Sketchpad
 - Ivan Sutherland - Head Mounted Display
 - Quest: A Long Ray's Journey Into Light